

Runtime Analysis

Table 1. Vector Runtime and Analysis

Operation	Cost	Executions	Total Cost
Open File	1	1	1
Read Line	1	n	n
Parse Line	1	n	n
Check Formatting	1	n	n
Create Course Object	1	n	n
Validate Prerequisites	1	n	n
Insert into Vector	1	n	n
Close File	1	1	1

Worst-Case Big O: $O(n)$

The vector processes each course one time while reading the file, validating prerequisites, creating the course object, and inserting it into the structure. Each of these operations occurs once for every course in the input file. Since insertion at the end of a vector occurs in $O(1)$ time and each course is processed once, the overall worst-case runtime for loading the data is $O(n)$.

Table 2. Hash Table Runtime Analysis

Operation	Cost	Executions	Total Cost
Open File	1	1	1
Read Line	1	n	n
Parse Line	1	n	n
Check Formatting	1	n	n
Create Course Object	1	n	n
Validate Prerequisites	1	n	n
Insert into Hash Table	n (Worst Case)	n	n^2
Close File	1	1	1

Worst-Case Big O: $O(n^2)$

The hash table processes each course one time while reading the file, validating prerequisites, creating the course object, and inserting it into the table. In the average case, insertion into a hash table occurs in $O(1)$ time. Although hash tables typically provide $O(1)$ insertion time, collisions can require traversing multiple entries in the worst case, resulting in $O(n)$ insertion time. Because n courses are inserted and each insertion may require $O(n)$ time in the worst case, the overall worst-case runtime becomes $O(n^2)$.

Table 3. Binary Search Tree Runtime Analysis

Operation	Cost	Executions	Total Cost
Open File	1	1	1
Read Line	1	n	n
Parse Line	1	n	n
Check Formatting	1	n	n
Create Course Object	1	n	n
Validate Prerequisites	1	n	n
Insert into Binary Search Tree	n (Worst Case)	n	n^2
Close File	1	1	1

Worst-Case Big O: $O(n^2)$

The binary search tree processes each course one time while reading the file, validating prerequisites, creating the course object, and inserting it into the tree. In the worst case, the tree may become unbalanced, causing each insertion to require traversing $O(n)$ nodes. Since n courses are inserted into the tree and each insertion may require $O(n)$ time, the overall worst-case runtime for loading the data is $O(n^2)$.

Table 4. Data Structure Comparison

Data Structure	Load File and Create Objects	Print Course List	Search for Course	Memory Usage
Vector	$O(n)$	$O(n \log n)$	$O(n)$	$O(n)$
Hash Table	$O(n)$ average, $O(n^2)$ worst case (collisions)	$O(n \log n)$	$O(1)$ average, $O(n)$ worst case	$O(n)$
Binary Search Tree	$O(n \log n)$ average, $O(n^2)$ worst case (unbalanced tree)	$O(n)$	$O(\log n)$ average, $O(n)$ worst case	$O(n)$

The runtime analysis shows that all three data structures require $O(n)$ memory because each course is stored only once. The vector provides the most efficient worst-case loading performance with a runtime of $O(n)$ because each course is processed and inserted one time. The hash table offers the fastest average-case search performance with $O(1)$ lookups, making it efficient for retrieving individual course information. The binary search tree provides $O(\log n)$ average-case search performance while maintaining courses in sorted order. This allows the course list to be displayed through an in-order traversal without requiring an additional sorting step.

Advantages and Disadvantages

Vector

The vector is simple to implement and efficient for loading data because new courses can be added directly to the end of the structure. It also uses $O(n)$ memory, since each course is stored only once. Searching for a specific course requires a linear scan through the vector, resulting in $O(n)$ search performance. In addition, the courses must be sorted before they can be printed in alphanumeric order. Because the advising program requires both course lookups and an ordered course list, the vector is not the most efficient choice.

Hash Table

The hash table provides very fast average-case search performance because courses can be accessed directly using the course number as a key. This makes retrieving information about an individual course efficient. The hash table also requires $O(n)$ memory. A disadvantage is that courses are not stored in sorted order, so an additional sorting step is required before printing the course list in alphanumeric order. While the hash table performs well for searching, the extra processing needed for sorting makes it less efficient for the advisor's requirement to display an ordered list of courses.

Binary Search Tree

The binary search tree maintains courses in sorted order as they are inserted into the structure. This allows the course list to be displayed in alphanumeric order through an in-order traversal without requiring an additional sorting step. The tree also provides efficient average-case search performance when locating a specific course and its prerequisites. A disadvantage is that performance can decrease if the tree becomes unbalanced, resulting in slower search and insertion operations. Even with this limitation, the binary

search tree provides a strong balance between efficient searching and maintaining sorted data. Because the advising program requires both course lookups and an ordered course list, the binary search tree is well suited for the application.

Recommendation

Based on the runtime analysis and the advisor's requirements, the binary search tree is the recommended data structure for the ABCU advising program. The vector provides efficient loading performance with a worst-case runtime of $O(n)$, but it requires a linear search to locate individual courses and an additional sorting step before displaying courses in alphanumeric order. The hash table offers the fastest average-case search performance with $O(1)$ lookups, but it also requires additional processing to sort courses before printing the course list.

The binary search tree provides a balance between efficient searching and maintaining data in sorted order. Courses are stored according to their course numbers, allowing an in-order traversal to display the course list in alphanumeric order without requiring a separate sorting operation. The tree also provides efficient average-case search performance when retrieving information about individual courses and their prerequisites.

Although the hash table offers faster average-case lookups, the binary search tree better satisfies the overall requirements of the advising program because it supports both efficient course searches and the ability to display courses in sorted order. For these reasons, the binary search tree is the best overall choice for this application.

References

GeeksforGeeks. (n.d.). *Analysis of algorithms | Big-O analysis*.
<https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/>

Southern New Hampshire University. (n.d.). CS 300 course materials.

Southern New Hampshire University. (n.d.). CS 300 pseudocode document.